vlcrlib — Cheat Sheet

Быстрый справочник по публичному API. R — right-handed Y-up, метры; Vulkan depth 0..1.

Vulkan 1.4 · C API
raylib-style · SDL3
PBR · IBL · GUI · Physics

МИНИМАЛЬНОЕ ПРИЛОЖЕНИЕ

```
#include <vlcrlib.h>
#include <vloaders.h>   /* FBX/OBJ/USD, LoadModelAuto */
#include <SDL3/SDL.h>

int main(void) {
    if (!InitWindow(1280, 720, "vlcrlib")) return 1;
    SetTargetFPS(60);

    Camera3D cam = { .position={0,1.5f,4}, .target={0,0.5f,0},
                     .up={0,1,0}, .fovy=45,
                     .nearPlane=0.05f, .farPlane=100 };

    while (!WindowShouldClose()) {
        BeginDrawing();
        ClearBackground(RAYWHITE);

        BeginMode3D(cam);
        DrawGrid(10, 1.0f);
        DrawCube((Vector3){0,0.5f,0}, (Vector3){1,1,1}, RED);
        EndMode3D();

        DrawText("Hello, vlcrlib", 20, 20, 20, DARKGRAY);
        EndDrawing();
    }
    CloseWindow();
    return 0;
}
```

C: compound literals `(Vector3){...}`. C++: `Vector3{...}` / `Camera3D{...}`.

ОКНО · КАДР · ВРЕМЯ

`InitWindow(w,h,title)`	bool; создаёт окно + Vulkan
`CloseWindow()`	освободить всё
`WindowShouldClose()`	цикл / Esc
`BeginDrawing()` / `EndDrawing()`	обрамляют кадр
`ClearBackground(Color)`	очистка
`SetTargetFPS(n)`	0 = без лимита
`GetFrameTime()`	float, секунды
`GetTime()`	double, с запуска
`GetFPS()`	int
`SetWindowTitle/Size/Position`	управление окном
`ToggleFullscreen()`	fullscreen/windowed
`IsWindowResized()`	bool

```
float dt = GetFrameTime();
double t  = GetTime();
```

ВВОД

`IsKeyDown(sc)`	удержание
`IsKeyPressed(sc)`	нажатие в кадре
`IsKeyReleased(sc)`	отпускание
`IsMouseButtonDown(b)`	1 L, 2 M, 3 R
`GetMousePosition()`	Vector2
`GetMouseWheelMove()`	float
`SetMousePosition(x,y)`	курсор

```
if (IsKeyDown(SDL_SCANCODE_W))
    cam.position.z -= 5.0f * dt;
if (IsMouseButtonPressed(MOUSE_BUTTON_LEFT)) { }
Vector2 m = GetMousePosition();
```

Клавиши — **SDL scancodes** (`SDL_SCANCODE_A`), не буквы. Подключи `<SDL3/SDL_scancode.h>`.

2D-РИСОВАНИЕ

`DrawPixel(x,y,c)`	точка
`DrawLine(x0,y0,x1,y1,c)`	линия
`DrawLineEx(a,b,thick,c)`	толстая
`DrawCircle(cx,cy,r,c)`	круг
`DrawCircleV(center,r,c)`	Vector2
`DrawCircleLines(cx,cy,r,c)`	контур
`DrawRectangle(x,y,w,h,c)`	прямоугольник
`DrawRectangleRec(rec,c)`	Rectangle
`DrawRectanglePro(rec,origin,rot,c)`	поворот
`DrawRectangleLines(x,y,w,h,c)`	рамка
`DrawTriangle(a,b,c,col)`	треугольник
`DrawPoly(center,sides,r,rot,c)`	полигон

```
DrawRectangle(20, 20, 200, 60, BLUE);
DrawRectangleLines(20, 20, 200, 60, WHITE);
DrawCircleV((Vector2){120,140}, 40, RED);
```

ТЕКСТ И ШРИФТЫ

`DrawText(txt,x,y,size,c)`	встроенный шрифт
`DrawTextEx(font,txt,pos,size,spacing,c)`	свой шрифт
`MeasureTextEx(font,txt,size,spacing)`	Vector2
`MeasureText(txt,size)`	int width
`LoadFont(path)`	Font
`LoadFontEx(path,size,first,count)`	Font
`UnloadFont(font)`	освободить

```
Font f = LoadFontEx("C:/Windows/Fonts/segoeui.ttf",
                    32, 32, 96);
DrawTextEx(f, "Score: 42", (Vector2){20,80}, 24, 1, WHITE);
UnloadFont(f);
```

VGui рисует текст только при `font.ok`. Если передать `(Font){0}`, кнопки будут без подписей.

ТЕКСТУРЫ И ИЗОБРАЖЕНИЯ

`LoadImage(path)`	CPU Image
`LoadImageFromMemory(...)`	из памяти
`LoadTexture(path)`	GPU Texture2D
`LoadTextureKTX2(path)`	BC7/BC5 KTX2
`LoadTextureFromImage(img)`	RGBA/SRGB
`LoadTextureFromImageSRGB(img)`	sRGB
`LoadTextureFromImageEx(img,opts)`	sampler+mips
`UnloadTexture(t)`	освободить
`UpdateTexture(t,pixels)`	обновить

```
Texture2D tex = LoadTexture("brick.png");
Texture2D bc = LoadTextureKTX2("brick.ktx2"); // BC7/BC5
DrawTexture(tex, 20, 20, WHITE);
UnloadTexture(bc);
UnloadTexture(tex);

TextureOptions o = { .srgb=1, .mipmaps=1,
                     .minFilter=5, .wrapS=0, .wrapT=0 };
Texture2D t2 = LoadTextureFromImageEx(img, &o);
```

Цветовые карты — sRGB; normal / metallic-roughness / AO — UNORM.

КАМЕРЫ · 2D / 3D

```
Camera2D c2 = { .offset={0,0}, .target={0,0},
                .rotation=0, .zoom=1 };
BeginMode2D(c2);
DrawCircleV((Vector2){0,0}, 50, RED);
EndMode2D();

Camera3D c3 = { .position={2,2,4}, .target={0,0,0},
                .up={0,1,0}, .fovy=45,
                .nearPlane=0.05f, .farPlane=100 };
BeginMode3D(c3);
DrawCube((Vector3){0,0,0}, (Vector3){1,1,1}, BLUE);
EndMode3D();
```

МЕШИ И МОДЕЛИ

`GenMeshCube(w,h,d)`	Mesh
`GenMeshSphere(r,rings,slices)`	Mesh
`GenMeshPlane(w,d,rx,rz)`	Mesh
`GenMeshCylinder(r,h,slices)`	Mesh
`GenMeshTorus(r1,r2,rs,ts)`	Mesh
`DrawMesh(mesh,tex,color,model)`	Matrix model
`LoadModelFromMesh(mesh)`	Model
`DrawModel(m,pos,rot,scale,tint)`	rot в радианах
`UnloadModel(m)`	освободить

```
Mesh cube = GenMeshCube(1,1,1);
Model m = LoadModelFromMesh(cube);
DrawModel(m, (Vector3){0,1,0}, (Vector3){0,0,0},
          (Vector3){1,1,1}, WHITE);
UnloadModel(m);
```

МАТЕРИАЛЫ PBR

```
Material mat = LoadMaterialDefault();
SetMaterialColor(&mat, WHITE);
SetMaterialBaseColorFactor(&mat, 1,1,1,1);
SetMaterialPBR(&mat, 0.0f, 0.35f);      // metallic, roughness
SetMaterialTexture(&mat, MATERIAL_MAP_ALBEDO, albedo);
SetMaterialTexture(&mat, MATERIAL_MAP_NORMAL, normal);
SetMaterialTexture(&mat, MATERIAL_MAP_METALLICROUGHNESS, mr);
SetMaterialTexture(&mat, MATERIAL_MAP_AO, ao);
SetMaterialEmission(&mat, WHITE, 1.0f);
SetMaterialIOR(&mat, 1.5f);
```

`MATERIAL_MAP_ALBEDO`	0 sRGB
`MATERIAL_MAP_NORMAL`	1 UNORM
`MATERIAL_MAP_METALLICROUGHNESS`	2 G=rough B=metal
`MATERIAL_MAP_AO`	3 R=occlusion
`MATERIAL_MAP_EMISSION`	4 sRGB
`MATERIAL_MAP_SPECULAR`	5 A=weight
`MATERIAL_MAP_CLEARCOAT`	6 R/G

ОСВЕЩЕНИЕ

```
vgraphics_SetDirectionalLight(true,
    (Vector3){-0.3f,-0.7f,-0.6f}, WHITE, 3.0f);
vgraphics_SetAmbientLight(true, (Color){90,100,120,255}, 0.15f);
vgraphics_SetPointLight(true, (Vector3){2,3,2},
    (Color){255,200,120,255}, 5.0f, 10.0f);
vgraphics_SetSpotLight(true, pos, dir, WHITE,
    8.0f, 15.0f, 0.9f, 0.7f);
vgraphics_SetAreaLight(true, pos, normal,
    2.0f, 1.0f, WHITE, 4.0f);
vgraphics_SetFog(true, GRAY, 5.0f, 40.0f, 0.0f);
```

```
VLightData lights[64];
lights[0] = VLightDataDirectional(dir, WHITE, 3.0f);
vgraphics_SetLightArray(lights, 1);
```

> SSBO-путь: `VLCRLIB_MAX_GPU_LIGHTS = 64`.

IBL · ТЕНИ · ПОСТОБРАБОТКА

```
vgraphics_LoadIBL(NULL);          // процедурная студия
vgraphics_LoadIBL("sky.hdr");     // HDR
vgraphics_SetIBLIntensity(1.2f);
vgraphics_SetSkybox(1);

vgraphics_SetShadows(true, 2048, 0.0015f, 0.8f);
vgraphics_SetShadowSoftness(1.5f);

SetSSAO(1, 0.5f, 0.6f);
SetBloom(0.9f, 0.15f);
SetGraphicsExposure(1.0f);
SetPostProcess(1);
SetFXAA(1);   // or:
SetTAA(1);    // jitter + history + velocity/reprojection
```

> IBL загружай **между** BeginDrawing/EndDrawing: пересоздаются descriptor-bound images.

АТМОСФЕРА

```
AtmosphereParams a = {0};
vgraphics_GetAtmosphere(&a);
a.enabled = 1;
a.timeOfDay = 18.5f;
a.cloudCoverage = 0.3f;
a.haze = 0.2f;
a.starIntensity = 0.4f;
vgraphics_SetAtmosphere(&a);
```

DEBUG VIEWS

```
SetGraphicsDebugView(0); // final
SetGraphicsDebugView(1); // albedo
SetGraphicsDebugView(2); // normal
SetGraphicsDebugView(3); // roughness
SetGraphicsDebugView(4); // metallic
SetGraphicsDebugView(5); // AO
SetGraphicsDebugView(6); // emissive
SetGraphicsDebugView(7); // shadow

int v = GetGraphicsDebugView();
SetGraphicsDebugLog(1);
```

АНИМАЦИЯ / СКИНИНГ

```
SkinnedModel sk = LoadSkinnedModel("hero.glb");
int n = GetSkinnedModelAnimationCount(sk);
for (int i=0;i<n;i++) {
    Animation a = GetSkinnedModelAnimation(sk,i);
    const char *name = GetSkinnedModelAnimationName(sk,i);
    float dur = GetSkinnedModelAnimationDuration(sk,i);
}
DrawSkinnedModel(sk, GetSkinnedModelAnimation(sk,0),
                 t, (Vector3){0}, (Vector3){0},
                 (Vector3){1,1,1}, WHITE);

// crossfade idle -> walk
DrawSkinnedModelBlend(sk, idle, ti, walk, tw, 0.5f,
    (Vector3){0}, (Vector3){0}, (Vector3){1,1,1}, WHITE);

// state machine (vanimator.h)
Animator *an = CreateAnimator(sk);
AnimatorAddState(an, "idle", idle, true);
AnimatorAddState(an, "walk", walk, true);
AnimatorPlay(an, "idle", 0.2f);
AnimatorAddTransition(an, "idle", "walk",
                      "speed", ANIM_COND_GREATER, 0.5f, 0.2f);
AnimatorSetFloat(an, "speed", 1.0f);
AnimatorSetOverlay(an, "idle", 0.35f);
// or additive relative to bind pose:
AnimatorSetAdditiveOverlay(an, "breath", 0.5f);
AnimatorAddEvent(an, "walk", 0.5f, "footstep");
// each frame:
const char *ev;
while ((ev = AnimatorNextEvent(an)) != NULL)
    printf("%s\n", ev);
AnimatorSetRootMotion(an, true);
AnimatorUpdate(an, dt);
pos = Vector3Add(pos, AnimatorGetRootMotion(an));
AnimatorDraw(an, pos, (Vector3){0},
             (Vector3){1,1,1}, WHITE);

// pose + IK (bone chain upper->mid->end)
BonePose pose[256];
GetAnimationPose(idle, 0.0f, pose, 256);
ApplyTwoBoneIkToPose(sk, pose, 256, 2, 3, 4, handTarget, pole);
DrawSkinnedModelPose(sk, pose, 256, pos, (Vector3){0},
                     (Vector3){1,1,1}, WHITE);
UnloadSkinnedModel(sk);
```

```
int bones = GetSkinnedModelBoneCount(sk);
for (int b=0;b<bones;b++)
    printf("%d %s\n", b,
           GetSkinnedModelBoneName(sk,b));
```

Создать клип

```
AnimTrackData *tracks = calloc(bones, sizeof(*tracks));
AnimQuatKey rot[3] = {
  {0.0f, {0,0,0,1}},
  {0.5f, {0,0.087f,0,0.996f}},
  {1.0f, {0,0,0,1}}
};
tracks[spine].rot = rot;
tracks[spine].rotCount = 3;
int clip = vgraphics_AddSkinnedAnimation(sk, "idle",
                                         1.0f, tracks, bones);
free(tracks);
```

GUI (VGUI)

```
vtext_EnsureDefaultFont();
Font font = GetFontDefault();
VGuiContext gui = {0};
gui.theme = VGuiDefaultTheme();

// каждый кадр:
VGuiBeginFrame(&gui, font, GetFrameTime());
if (VGuiButton(&gui, (Rectangle){20,20,120,28}, "Open"))
    open_file();
VGuiCheckBox(&gui, (Rectangle){20,56,150,22}, &loop, "Loop");
VGuiSlider(&gui, (Rectangle){20,84,200,18}, &speed, 0, 3);
VGuiInputText(&gui, (Rectangle){20,110,300,26}, path, 512);
VGuiEndFrame(&gui);
```

`VGuiLabel/Text`	подписи
`VGuiButton/SmallButton`	кнопки
`VGuiCheckBox/Radio`	флаги
`VGuiSlider/Int`	float/int
`VGuiCombo/ListBox/Tabs`	выбор
`VGuiColorPicker`	Color*
`VGuiBeginWindow`	окно

АУДИО

```
InitAudioDevice();

Sound s = LoadSound("hit.wav");
PlaySound(s);
SetSoundVolume(s, 0.8f);

Music m = LoadMusicStream("music.ogg");
PlayMusicStream(m);
SetMusicVolume(m, 0.7f);
while (IsMusicStreamPlaying(m))
    UpdateMusicStream(m);

UnloadMusicStream(m);
CloseAudioDevice();

// positional mixer (miniaudio)
InitAudio3D();
LoadReverbIR("room_ir.wav", 0.35f); // before sounds
Sound3D s3 = LoadSound3D("hit.wav");
SetListener3D((Vector3){0,1,0},
              (Vector3){0,0,-1}, (Vector3){0,1,0});
PlaySound3D(s3, (Vector3){2,1,-4});
SetSound3DPosition(s3, (Vector3){3,1,-2});
SetSound3DOcclusion(s3, 0.5f);   // 0..1, includes low-pass
SetSound3DReverb(s3, 0.4f);      // feedback-delay zone
SetSound3DVelocity(s3, (Vector3){1,0,0});
SetListener3DVelocity((Vector3){0,0,0});

Sound3D music = LoadMusic3D("theme.ogg"); // streamed
SetSound3DLooping(music, true);
PlaySound3D(music, (Vector3){0,0,0});
PauseSound3D(music);
ResumeSound3D(music);
SeekSound3D(music, 12.0f);
float t = GetSound3DTime(music);
float len = GetSound3DDuration(music);
UnloadSound3D(music);
UnloadSound3D(s3);
CloseAudio3D();
```

> VLC-аудио — `vaudio.h`; 3D-микшер — `vaudio3d.h` (miniaudio).

ЗАГРУЗКА МОДЕЛЕЙ

> Для FBX/OBJ/USD и `LoadModelAuto` подключи `#include <vloaders.h>`.

`LoadModelGLTFTiny(path)`	.glb/.gltf (tinygltf)
`LoadSkinnedModel(path)`	glTF со скелетом
`LoadFBX(path)`	Assimp, статический
`LoadFBXSkinned(path)`	Assimp + клипы
`LoadOBJ(path)`	Assimp/OBJ+MTL
`LoadUSD(path)`	OpenUSD, статический
`LoadModelAuto(path)`	по расширению
`LoadCookedModel(path)`	.vmesh v1/v2/v3
`LoadTextureKTX2(path)`	BC7/BC5 + mips

```
Model m = LoadModelAuto("chair.obj");
Model c = LoadCookedModel("cooked/chair.vmesh");
Model u = LoadUSD("scene.usdz"); // нужен OpenUSD
Model f = LoadFBX("robot.fbx");  // нужен Assimp

Scene sc; SceneInit(&sc);
SceneLoad(&sc, "level.scene.json");
SceneRuntime rt;
SceneLoadModels(&sc, "level.scene.json", &rt);
SceneDraw(&sc, &rt);
SceneUnloadModels(&rt);
```

> USD сейчас без анимаций и без полного `UsdPreviewSurface`. FBX нужны DLL Assimp: `assimp-vc145-mt`, `poly2tri`, `minizip`, `z`, `kubazip`, `pugixml`.

МАТЕМАТИКА

```
#define VLCRLIB_PI 3.14159265f
Vector3 a = {1,2,3}, b = {4,5,6};
Vector3 c = Vector3Add(a,b);
Vector3 n = Vector3Normalize(a);
float d  = Vector3Dot(a,b);
Vector3 x = Vector3Cross(a,b);
Vector3 l = Vector3Lerp(a,b,0.5f);

// two-bone IK: root, mid, end -> target, pole
Vector3 ikMid, ikEnd;
Vector3SolveTwoBoneIK(root, mid, end,
                      target, (Vector3){0,0,1},
                      &ikMid, &ikEnd);

Matrix model = MatrixMultiply(
    MatrixTranslate(0,1,0),
    MatrixRotateY(VLCRLIB_PI/4));
Matrix view = MatrixLookAt(eye, target, (Vector3){0,1,0});
Matrix proj = MatrixPerspective(
    45*VLCRLIB_DEG2RAD, 16.0f/9.0f, 0.05f, 1000);
Quaternion q = QuaternionFromAxisAngle(
    (Vector3){0,1,0}, VLCRLIB_PI/2);
Matrix rot = QuaternionToMatrix(q);
```

ЦВЕТА

```
Color c = {255, 128, 0, 255};
// WHITE BLACK RED GREEN BLUE YELLOW
// ORANGE PINK PURPLE VIOLET
// LIGHTGRAY GRAY DARKGRAY
// SKYBLUE DARKBLUE MAROON LIME
// BEIGE BROWN DARKBROWN GOLD
// RAYWHITE BLANK MAGENTA
```

RENDER TEXTURE

```
RenderTexture2D rt = LoadRenderTexture(1280, 720);
BeginTextureMode(rt);
ClearBackground(BLACK);
DrawCircleV((Vector2){640,360}, 120, RED);
EndTextureMode();

DrawTextureRec(rt.texture,
    (Rectangle){0,0,1280,-720},
    (Vector2){0,0}, WHITE);
UnloadRenderTexture(rt);
```

ТИПЫ И ХЭНДЛЫ

```
typedef struct Model {
    unsigned id;
    int meshCount, materialCount;
} Model;

typedef struct Mesh {
    unsigned id;
    int vertexCount, indexCount;
} Mesh;

typedef struct Texture2D {
    unsigned id;
    int width, height, mipmaps, format;
} Texture2D;

typedef struct Material {
    unsigned id;
    Color baseColor;
    float baseColorFactor[4];
    float metallic, roughness, ior;
    float normalScale, aoStrength;
    Color emissionColor;
    float emissionStrength;
    int alphaMode;      // 0 OPAQUE 1 MASK 2 BLEND
    float alphaCutoff;
    int doubleSided;
    Texture2D albedoMap, normalMap;
    Texture2D metallicRoughnessMap;
    Texture2D aoMap, emissionMap;
} Material;
```

FRAMESTATS

```
FrameStats fs;
vgraphics_GetFrameStats(&fs);

printf("%d draws, %llu tris, %.2f ms\n",
       fs.drawCalls,
       (unsigned long long)fs.triangles,
       fs.cpuFrameMs);

// fs.shadowDraws, fs.transparentDraws, fs.culledDraws
// fs.gpuFrameMs, fs.shadowMs, fs.sceneMs, fs.postMs
// fs.textureCount, fs.meshCount

// validation: set VLCRLIB_VALIDATION=1
int verr = vgraphics_GetValidationErrorCount();

// outside BeginDrawing/EndDrawing:
vgraphics_CaptureScreenshot("frame.png");
```

ФИЗИКА 2D / 3D

2D (Box2D)

```
PhysicsWorld2D *w = CreatePhysicsWorld2D((Vector2){0,9.81f});
PhysicsBody2D *b = CreatePhysicsBody2DBox(w,
    (Vector2){0,0}, (Vector2){0.5f,0.5f},
    PHYS_BODY_DYNAMIC2D, 1.0f, 0.5f, 0.2f);

StepPhysicsWorld2D(w, dt);
Vector2 p = GetPhysicsBody2DPosition(b);
DestroyPhysicsBody2D(b);
DestroyPhysicsWorld2D(w);
```

3D (Bullet)

```
PhysicsWorld3D *w = CreatePhysicsWorld3D(
    VLCRLIB_DEFAULT_GRAVITY);
PhysicsBody3D *b = CreatePhysicsBody3D(w,
    PHYS_SHAPE_BOX3D, (Vector3){0,5,0},
    (Vector3){1,1,1}, PHYS_BODY_DYNAMIC3D,
    1.0f, 0.5f, 0.2f);

StepPhysicsWorld3D(w, dt);
Vector3 p = GetPhysicsBody3DPosition(b);
```

> Jolt-бэкенд: `Jolt_Init()`, `Jolt_CreateWorld()`, `Jolt_Step()`, `Jolt_CreateBody()`.

СБОРКА

```
cmake -S . -B build -G "Visual Studio 18 2026" -A x64 `
  -DCMAKE_TOOLCHAIN_FILE=C:/vcpkg/scripts/buildsystems/vcpkg.cmake

cmake --build build --config Release
```

Зависимости: Vulkan SDK, SDL3, tinygltf; опционально Assimp, OpenUSD, Jolt, VLC.

```
build/example/Release/model_animation_viewer.exe
```

ЧАСТЫЕ ОШИБКИ

- **Чёрный экран 3D:** не вызван `BeginMode3D`; камера внутри объекта; near/far.
- **Модель гигантская:** FBX в сантиметрах — масштаб `0.01`.
- **Текстура полосами:** RGB залит как RGBA / неверный `bufferRowLength`.
- **GUI без текста:** `VGuiBeginFrame(...,(Font){0},...)`.
- **Нет анимации:** в FBX нет костей (`bones=0`) или клип не добавлен.
- **IBL ломает кадр:** загрузка внутри BeginDrawing.
- **USD не грузится:** не собран OpenUSD, `LoadUSD` — stub.